

ASSIGNMENT – 02

SQL DDL, Constraints, Queries and Triggers

Order Processing, Event–Prize and Student Information Systems

Name	M. Yogeshwaran
Register Number	192524363
Course Code	CSA0507
Course Name	Data Base Management System
Assignment	Assignment – 02

Database Management System – SQL Implementation Assignment

Question 1 & 2 — Order Processing Database

1. Schema Description

The Order Processing database consists of four related tables — Customer, Cust_Order, Item and Order_item — capturing customers, the orders they place, the items available for sale, and the line items within each order (a many-to-many relationship between orders and items resolved by Order_item).

Table	Attributes	Constraints
Customer	Customerno VARCHAR2(5) — PK, Cname VARCHAR2(50)	Customerno must start with 'C'
Cust_Order	Orderno VARCHAR2(5) — PK, Odate DATE, Customerno — FK → Customer, Ord_amt NUMBER(8)	Orderno must start with 'O' ; Ord_amt is a derived attribute, DEFAULT 0
Item	Itemno VARCHAR2(5) — PK, Item_name VARCHAR2(30), unit_price NUMBER(5)	Itemno must start with 'I'
Order_item	Orderno — FK → Cust_Order, Itemno — FK → Item, qty NUMBER(3)	Composite PK (Orderno, Itemno)

Relationship Overview

Customer (1) — (M) Cust_Order : one customer can place many orders, each order belongs to exactly one customer.
Cust_Order (1) — (M) Order_item (M) — (1) Item : each order can contain many items and each item can appear in many orders, resolved through the associative entity Order_item.

A. DDL — Primary Key, Check and Foreign Key Constraints

```
CREATE TABLE Customer (  
  Customerno VARCHAR2(5) PRIMARY KEY,  
  Cname VARCHAR2(50) NOT NULL,  
  CONSTRAINT chk_cust_no CHECK (Customerno LIKE 'C%')  
);  
  
CREATE TABLE Cust_Order (  
  Orderno VARCHAR2(5) PRIMARY KEY,  
  Odate DATE,  
  Customerno VARCHAR2(5) REFERENCES Customer(Customerno),  
  Ord_amt NUMBER(8) DEFAULT 0,  
  CONSTRAINT chk_order_no CHECK (Orderno LIKE 'O%')  
);  
  
CREATE TABLE Item (  
  Itemno VARCHAR2(5) PRIMARY KEY,  
  Item_name VARCHAR2(30) NOT NULL,  
  unit_price NUMBER(5),  
  CONSTRAINT chk_item_no CHECK (Itemno LIKE 'I%')  
);  
  
CREATE TABLE Order_item (  
  Orderno VARCHAR2(5) REFERENCES Cust_Order(Orderno),  
  Itemno VARCHAR2(5) REFERENCES Item(Itemno),  
  qty NUMBER(3),  
  CONSTRAINT pk_order_item PRIMARY KEY (Orderno, Itemno)  
);
```

chk_cust_no, chk_order_no and chk_item_no enforce the required starting letters 'C', 'O' and 'I'. Ord_amt defaults to 0 as it is a derived attribute (to be updated by a trigger/application logic that sums qty × unit_price for the order's items).

Sample Data (used to illustrate the queries below)

```
-- Customer
INSERT INTO Customer VALUES ('C001','Ravi Kumar');
INSERT INTO Customer VALUES ('C002','Anitha Sharma');
INSERT INTO Customer VALUES ('C003','Suresh Babu');
INSERT INTO Customer VALUES ('C004','Divya Rani');

-- Item
INSERT INTO Item VALUES ('I001','Laptop',45000);
INSERT INTO Item VALUES ('I002','Mouse',500);
INSERT INTO Item VALUES ('I003','Keyboard',800);
INSERT INTO Item VALUES ('I004','Monitor',9000);
INSERT INTO Item VALUES ('I005','Webcam',1800);
INSERT INTO Item VALUES ('I006','Headset',1200);

-- Cust_Order (Ord_amt shown after being computed/updated)
INSERT INTO Cust_Order VALUES ('0001', DATE '2026-01-05', 'C001', 46300);
INSERT INTO Cust_Order VALUES ('0002', DATE '2026-01-10', 'C001', 54000);
INSERT INTO Cust_Order VALUES ('0003', DATE '2026-01-15', 'C001', 2700);
INSERT INTO Cust_Order VALUES ('0004', DATE '2026-01-20', 'C001', 1600);
INSERT INTO Cust_Order VALUES ('0005', DATE '2026-02-02', 'C002', 54000);
INSERT INTO Cust_Order VALUES ('0006', DATE '2026-02-08', 'C003', 4800);
INSERT INTO Cust_Order VALUES ('0007', DATE '2026-02-11', 'C003', 46300);
INSERT INTO Cust_Order VALUES ('0008', DATE '2026-02-15', 'C004', 3600);

-- Order_item
INSERT INTO Order_item VALUES ('0001','I001',1);
INSERT INTO Order_item VALUES ('0001','I002',2);
INSERT INTO Order_item VALUES ('0001','I003',1);
INSERT INTO Order_item VALUES ('0002','I004',1);
INSERT INTO Order_item VALUES ('0002','I005',1);
INSERT INTO Order_item VALUES ('0003','I002',3);
INSERT INTO Order_item VALUES ('0003','I006',1);
INSERT INTO Order_item VALUES ('0004','I003',2);
INSERT INTO Order_item VALUES ('0005','I001',1);
INSERT INTO Order_item VALUES ('0005','I004',1);
INSERT INTO Order_item VALUES ('0006','I005',2);
INSERT INTO Order_item VALUES ('0006','I006',1);
INSERT INTO Order_item VALUES ('0007','I001',1);
INSERT INTO Order_item VALUES ('0007','I002',1);
INSERT INTO Order_item VALUES ('0007','I003',1);
INSERT INTO Order_item VALUES ('0008','I006',3);
```

B. Customers Who Have Placed More Than 3 Orders

```
SELECT c.Customerno, c.Cname, COUNT(o.Orderno) AS Total_Orders
FROM   Customer c JOIN Cust_Order o ON c.Customerno = o.Customerno
GROUP BY c.Customerno, c.Cname
HAVING COUNT(o.Orderno) > 3;
```

This groups orders by customer and keeps only the groups whose order count exceeds 3.

Output

CUSTOMERNO	CNAME	TOTAL_ORDERS
C001	Ravi Kumar	4

Only Ravi Kumar (C001) has placed more than 3 orders in the sample data (4 orders); every other customer placed 3 or fewer and is excluded by the HAVING clause.

C. Items Priced Below the Average Item Price of Their Own Order

```
SELECT oi.Orderno, i.Itemno, i.Item_name, i.unit_price
FROM   Order_item oi
       JOIN Item i ON oi.Itemno = i.Itemno
WHERE  i.unit_price < (
        SELECT AVG(i2.unit_price)
        FROM   Order_item oi2
               JOIN Item i2 ON oi2.Itemno = i2.Itemno
               WHERE oi2.Orderno = oi.Orderno
      )
ORDER BY oi.Orderno, i.Itemno;
```

The correlated subquery computes the average unit_price of all items belonging to the same order (oi2.Orderno = oi.Orderno); the outer query then keeps only the items in that order whose price is below that average.

Output

ORDERNO	ITEMNO	ITEM_NAME	UNIT_PRICE
O001	I002	Mouse	500
O001	I003	Keyboard	800
O002	I005	Webcam	1800
O003	I002	Mouse	500
O005	I004	Monitor	9000
O006	I006	Headset	1200
O007	I002	Mouse	500
O007	I003	Keyboard	800

Orders O004 and O008 contain only a single item each, so that item equals (not less than) the order average and correctly does not appear in the result.

Question 3 — Trigger: Auto-Create Prizes for Every New Event

Schema Description

To support the requirement, two supporting tables are used: Event (one row per event) and Prizes (the prize positions and amounts awarded for an event). A row-level AFTER INSERT trigger on Event automatically inserts the three standard prize rows — 1st (1500), 2nd (1000) and 3rd (500) — into Prizes whenever a new event is created.

Table	Attributes	Constraints
Event	Event_no NUMBER — PK, Event_name VARCHAR2(50), Event_date DATE	Event_no generated by event_seq

Table	Attributes	Constraints
Prizes	Prize_no NUMBER — PK, Event_no — FK → Event, Position NUMBER(1), Amount NUMBER(6)	Position IN (1,2,3); Prize_no generated by prize_seq

DDL — Supporting Tables and Sequences

```
CREATE SEQUENCE event_seq START WITH 1 INCREMENT BY 1;
CREATE SEQUENCE prize_seq START WITH 1 INCREMENT BY 1;
```

```
CREATE TABLE Event (
  Event_no    NUMBER PRIMARY KEY,
  Event_name  VARCHAR2(50) NOT NULL,
  Event_date  DATE
);
```

```
CREATE TABLE Prizes (
  Prize_no    NUMBER PRIMARY KEY,
  Event_no    NUMBER REFERENCES Event(Event_no),
  Position    NUMBER(1) CHECK (Position IN (1,2,3)),
  Amount      NUMBER(6)
);
```

Trigger Definition

```
CREATE OR REPLACE TRIGGER trg_create_prizes
AFTER INSERT ON Event
FOR EACH ROW
BEGIN
  INSERT INTO Prizes (Prize_no, Event_no, Position, Amount)
  VALUES (prize_seq.NEXTVAL, :NEW.Event_no, 1, 1500);

  INSERT INTO Prizes (Prize_no, Event_no, Position, Amount)
  VALUES (prize_seq.NEXTVAL, :NEW.Event_no, 2, 1000);

  INSERT INTO Prizes (Prize_no, Event_no, Position, Amount)
  VALUES (prize_seq.NEXTVAL, :NEW.Event_no, 3, 500);
END;
/
```

Test Execution

```
INSERT INTO Event (Event_no, Event_name, Event_date)
VALUES (event_seq.NEXTVAL, 'Annual Sports Meet', DATE '2026-09-10');

SELECT * FROM Prizes WHERE Event_no = 1;
```

Output

PRIZE_NO	EVENT_NO	POSITION	AMOUNT
1	1	1	1500
2	1	2	1000

PRIZE_NO	EVENT_NO	POSITION	AMOUNT
3	1	3	500

Inserting a single row into Event automatically fired the trigger and created exactly 3 corresponding rows in Prizes for that event, confirming the trigger works as required.

Question 4 — Student Information System Schema

Schema Description

The schema models students, the branches they belong to, the courses offered under each branch, and the results students obtain in each subject.

Table	Attributes	Constraints
Student	enrol_no — PK, s_name, phone, gender, DOB	s_name NOT NULL, phone NOT NULL
Branch	branch_code — PK, branch_name, HOD_name	—
Course	Subject_code — PK, Subject_name, details, Branch_code — FK → Branch	Subject_name NOT NULL
Result	enrol_no — FK → Student, Subject_code — FK → Course, marks, res, rank	res IN ('A','B','C','D','P','F'); rank IN (1,2,3,0.6); composite PK (enrol_no, Subject_code)

DDL — Tables with the Required Constraints

```
CREATE TABLE Branch (
  branch_code  VARCHAR2(10) PRIMARY KEY,
  branch_name  VARCHAR2(50),
  HOD_name     VARCHAR2(50)
);

CREATE TABLE Student (
  enrol_no     VARCHAR2(10) PRIMARY KEY,
  s_name       VARCHAR2(50) NOT NULL,
  phone        VARCHAR2(15) NOT NULL,
  gender       VARCHAR2(1),
  DOB          DATE
);

CREATE TABLE Course (
  Subject_code VARCHAR2(10) PRIMARY KEY,
  Subject_name  VARCHAR2(50) NOT NULL,
  details       VARCHAR2(200),
  Branch_code   VARCHAR2(10) REFERENCES Branch(branch_code)
);

CREATE TABLE Result (
  enrol_no     VARCHAR2(10) REFERENCES Student(enrol_no),
  Subject_code  VARCHAR2(10) REFERENCES Course(Subject_code),
  marks        NUMBER(3),
  res          VARCHAR2(1),
  rank         NUMBER(3,1),
  CONSTRAINT pk_result PRIMARY KEY (enrol_no, Subject_code),
  CONSTRAINT chk_res  CHECK (res IN ('A','B','C','D','P','F')),
  CONSTRAINT chk_rank CHECK (rank IN (1,2,3,0.6))
);
```

(A) *s_name* and *phone* are declared NOT NULL on *Student*, and *Subject_name* is NOT NULL on *Course*. (B) *chk_rank* restricts *rank* to the four values specified in the question — 1, 2, 3 and 0.6. (C) *chk_res* restricts *res* to the six grade values A, B, C, D, P or F.

Constraint Verification

```
-- Valid row – accepted
INSERT INTO Result VALUES ('E001','CS101',85,'A',1);

-- Violates chk_res (invalid grade 'E') – rejected
INSERT INTO Result VALUES ('E002','CS101',40,'E',2);
-- ORA-02290: check constraint (CHK_RES) violated

-- Violates chk_rank (value 4 not allowed) – rejected
INSERT INTO Result VALUES ('E003','CS101',70,'B',4);
-- ORA-02290: check constraint (CHK_RANK) violated

-- Violates NOT NULL on s_name – rejected
INSERT INTO Student VALUES ('E004', NULL, '9876543210','F','12-JAN-2004');
-- ORA-01400: cannot insert NULL into ("S_NAME")
```

Result

Test Case	Constraint Checked	Outcome
INSERT ('E001',..., 'A',1)	chk_res, chk_rank	Accepted
INSERT ('E002',..., 'E',2)	chk_res	Rejected — ORA-02290
INSERT ('E003',..., 'B',4)	chk_rank	Rejected — ORA-02290
INSERT student with s_name = NULL	NOT NULL (s_name)	Rejected — ORA-01400

All four constraints — NOT NULL on *s_name*/*phone*/*Subject_name*, the restricted rank list, and the restricted grade list — were verified to behave as specified.